



Pandas

1. Introducción

Pandas es una biblioteca de código abierto para el lenguaje de programación Python, diseñada para facilitar la manipulación, organización, limpieza, transformación y análisis de datos. Es una herramienta fundamental en áreas como *Ciencia de Datos* y *Machine Learning*, especialmente cuando se trabaja con información estructurada o tabular, similar a una hoja de cálculo de Excel o a una tabla de una base de datos.

Pandas proporciona estructuras de datos rápidas, flexibles y de alto rendimiento que permiten trabajar de manera eficiente con grandes conjuntos de información. Sus dos estructuras principales son **Series**, utilizada para representar datos unidimensionales (1D), y **DataFrame**, que organiza los datos de forma bidimensional (2D) mediante filas y columnas. Estas estructuras permiten trabajar con diferentes tipos de información, incluyendo datos numéricos, categóricos y series temporales.

<https://pandas.pydata.org/docs/>

2. Estructuras de datos

Las estructuras de datos son formas de organizar y almacenar información para facilitar su acceso, manipulación y análisis. En Pandas, las dos estructuras principales son **Series** y **DataFrame**, diseñadas para trabajar con datos etiquetados de manera sencilla y eficiente.

2.1 Series

Una **Series** es una estructura de datos unidimensional (1D), similar a una columna de una tabla. Está formada por un conjunto de valores y un índice asociado a cada uno de ellos, lo que permite identificar y acceder a los datos mediante etiquetas.

Sintaxis:

```
pd.Series(data, index=None, dtype=None, name=None)
```

data: son los datos que se almacenarán.

index: etiquetas asociadas a cada dato.

dtype: tipo de dato de los elementos.

name: nombre que tendrá la Series.

EJEMPLO 1:

```
In [2]: import pandas as pd

# Crear una Series
presion = pd.Series([2500, 'dos', 2380, 2300, 2210])

print(presion)
```

```
0    2500
1      dos
2    2380
3    2300
4    2210
dtype: object
```

EJEMPLO 2:

```
In [3]: import pandas as pd

# Crear una Series
presion = pd.Series(
    [2500, 2450, 2380],
    index=["Pozo A", "Pozo B", "Pozo C"],
    dtype="float64",
    name="Presión"
)

print(presion)
```

```
Pozo A    2500.0
Pozo B    2450.0
Pozo C    2380.0
Name: Presión, dtype: float64
```

2.2 DataFrame

Un **DataFrame** es una estructura de datos bidimensional (2D) organizada en filas y columnas, similar a una hoja de cálculo de Excel o a una tabla de una base de datos. Cada columna de un **DataFrame** puede considerarse una **Series** y puede contener diferentes tipos de datos, como números, texto, fechas o valores booleanos.

Sintaxis

```
pd.DataFrame(data, index=None, columns=None, dtype=None)
```

data: contiene los datos que formarán el DataFrame.

index: define las etiquetas de las filas.

columns: define o selecciona las columnas.

dtype: permite especificar el tipo de dato.

EJEMPLO 1:

```
In [4]: import pandas as pd
datos = {
    "Pozo": ["Pozo A", "Pozo B", "Pozo C", "Pozo D"],
    "Profundidad_m": [2500, 2800, 3100, 2750],
    "Presion_psi": [3200, 3500, 3900, 3400],
    "Produccion_bpd": [850, 720, 950, 680]
}

df = pd.DataFrame(datos)
df
```

```
Out[4]:
```

	Pozo	Profundidad_m	Presion_psi	Produccion_bpd
0	Pozo A	2500	3200	850
1	Pozo B	2800	3500	720
2	Pozo C	3100	3900	950
3	Pozo D	2750	3400	680

NOTA: Algo importante es que cada columna de un DataFrame es una Series.

```
In [5]: df["Presion_psi"]
```

```
Out[5]: 0    3200
        1    3500
        2    3900
        3    3400
        Name: Presion_psi, dtype: int64
```

EJEMPLO 2:

```
In [6]: # DataFrame (En colab las listas siempre tienen que ser de la misma dimensión)
import pandas as pd
data = {
    "nombre": ["Ana", "Luis", "Pedro"],
    "edad": [23, 30, 28],
    "ciudad": ["Madrid", "Bogotá", "CDMX"]
}

df = pd.DataFrame(data)
print(type(df))
df
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Out[6]:
```

	nombre	edad	ciudad
0	Ana	23	Madrid
1	Luis	30	Bogotá
2	Pedro	28	CDMX

3. Exploración de datos

La exploración de datos es una de las primeras etapas al trabajar con un **DataFrame**, ya que permite conocer su estructura, dimensiones, nombres de las columnas, tipos de datos y algunas características estadísticas de la información.

Pandas proporciona diferentes métodos y atributos que permiten realizar una inspección inicial de los datos antes de comenzar con su selección, modificación o análisis.

El análisis exploratorio de un DataFrame puede ser realizado con los métodos siguientes:

- Visualizar las primeras filas **.head()**
- Visualizar las últimas filas **.tail()**
- Dimensiones **.shape**
- Nombres de las columnas **.columns**
- Información del DataFrame **.info()**
- Estadísticas descriptivas **.describe()**

```
In [7]: import pandas as pd
pokemon = pd.read_excel("N_Pokedex_Gen_9.xlsx")
```

pokemon

Out[7]:

	National Dex	Pokemon Name	Generation	Type	Kanto Dex # (RBYFRLG)	Kanto Dex # (LGPE)	#
0	1.0	Bulbasaur	1	Starter	1.0	1.0	
1	2.0	Ivysaur	1	Starter	2.0	2.0	
2	3.0	Venusaur	1	Starter	3.0	3.0	
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	
...	...	...	...	...	...	...	
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

202 rows × 66 columns



3.1 Visualizar las primeras filas

El método `.head()` permite visualizar las primeras filas de un DataFrame. Por defecto, Pandas muestra las primeras 5 filas, aunque es posible indicar entre paréntesis el número de filas que se desea visualizar.

Sintaxis:

`DataFrame.head(n)`

Donde n representa el número de filas que se desean mostrar.

EJEMPLO:

```
In [8]: # df.head() muestra los primeros cinco registros
pokemon.head()
```

Out[8]:

	National Dex	Pokemon Name	Generation	Type	Kanto Dex # (RBYFRLG)	Kanto Dex # (LGPE)	New #
0	1.0	Bulbasaur	1	Starter	1.0	1.0	2
1	2.0	Ivysaur	1	Starter	2.0	2.0	2
2	3.0	Venusaur	1	Starter	3.0	3.0	2
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	2

5 rows × 66 columns



3.2 Visualizar las últimas filas

El método `.tail()` permite visualizar las últimas filas de un DataFrame. De manera predeterminada muestra las últimas 5 filas, pero también se puede especificar la cantidad de filas.

Sintaxis:

`DataFrame.tail(n)`

EJEMPLO:

```
In [9]: # df.tail() muestra los últimos cinco registros
pokemon.tail()
```

Out[9]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	↑ #
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

5 rows × 66 columns



3.3 Dimensiones

El atributo `.shape` permite conocer las dimensiones de un DataFrame, es decir, el número de filas y columnas que contiene.

Sintaxis:

DataFrame.shape

EJEMPLO:

```
In [10]: # .shape indica el número de columnas y filas
pokemon.shape
```

Out[10]: (202, 66)

3.4 Nombres de las columnas

El atributo `.columns` permite consultar los nombres o etiquetas de todas las columnas que forman un DataFrame.

```
In [11]: # .columns
pokemon.columns
```

```
Out[11]: Index(['National\nDex', 'Pokemon\nName', 'Generation', 'Type',
               'Kanto Dex #\n(RBYFRLG)', 'Kanto Dex #\n(LGPE)', 'New Dex #\n(GSC)',
               'Johto Dex #\n(HGSS)', 'Hoenn Dex #\n(RSE)', 'Hoenn Dex #\n(ORAS)',
               'Sinnoh Dex #\n(DPBDSP)', 'Sinnoh Dex #\n(Pt)', 'Unova Dex #\n(BW)',
               'Unova Dex #\n(B2W2)', 'Central Kalos Dex #\n(XY)',
               'Coastal Kalos Dex #\n(XY)', 'Mount Kalos Dex #\n(XY)',
               'Alola Dex #\n(SM)', 'Melemele Dex #\n(SM)', 'Akala Dex #\n(SM)',
               'Ula'ula Dex #\n(SM)', 'Poni Dex #\n(SM)', 'Alola Dex #\n(USUM)',
               'Melemele Dex #\n(USUM)', 'Akala Dex #\n(USUM)',
               'Ula'ula Dex #\n(USUM)', 'Poni Dex #\n(USUM)', 'Galar Dex #\n(SwSh)',
               'Isle of Armor Dex #\n(SwSh)', 'Crown Tund Dex #\n(SwSh)',
               'Hisui Dex #\n(LA)', 'Paldea Dex #\n(SV)', 'Kitakami Dex # (SV)',
               'Blueberry Dex # (SV)', 'HP', 'Attack', 'Defense', 'Special\nAttack',
               'Special\nDefense', 'Speed', 'Total', 'Type I', 'Type II', 'Ability I',
               'Ability II', 'Hidden Ability', 'Height (ft)', 'Height (m)',
               'Weight (lbs)', 'Weight (kg)', 'HP EVs', 'Attcak EVs', 'Defense EVs',
               'Special Attack EVs', 'Special Defense EVs', 'Speed EVs',
               'Experience\nValue', 'Catch Rate', 'Experience\nGrowth',
               'Base Friendship', 'Pokedex Color', 'Gender Ratio', 'Egg Cycles',
               'Egg Group I', 'Egg Group II', 'Evolution\nMethod'],
              dtype='object')
```

3.5 Información del DataFrame

El método `.info()` proporciona un resumen general de la estructura del DataFrame.

Permite conocer el número de registros, los nombres de las columnas, la cantidad de valores no nulos, los tipos de datos y el uso aproximado de memoria.

Sintaxis:

```
DataFrame.info()
```

EJEMPLO:

```
In [12]: # .info() muestra una tabla completa del La cántidad de datos, tipo y columnas
          pokemon.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 202 entries, 0 to 201
Data columns (total 66 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   National                                202 non-null    float64
Dex
1   Pokemon                                202 non-null    object
Name
2   Generation                            202 non-null    int64
3   Type                                  76 non-null     object
4   Kanto Dex # (RBYFRLG)                  151 non-null    float64
5   Kanto Dex # (LGPE)                  151 non-null    float64
6   New Dex # (GSC)                    151 non-null    float64
7   Johto Dex # (HGSS)                  151 non-null    float64
8   Hoenn Dex # (RSE)                   47 non-null     float64
9   Hoenn Dex # (ORAS)                  50 non-null     float64
10  Sinnoh Dex # (DPBDS)                 32 non-null     float64
11  Sinnoh Dex # (Pt)                   46 non-null     float64
12  Unova Dex # (BW)                    0 non-null      float64
13  Unova Dex # (B2W2)                  38 non-null     float64
14  Central Kalos Dex # (XY)             44 non-null     float64
15  Coastal Kalos Dex # (XY)            41 non-null     float64
16  Mount Kalos Dex # (XY)              37 non-null     float64
17  Alola Dex # (SM)                   112 non-null    float64
18  Melemele Dex # (SM)                59 non-null     float64
19  Akala Dex # (SM)                   49 non-null     float64
20  Ula'ula Dex # (SM)                63 non-null     float64
21  Poni Dex # (SM)                   40 non-null     float64
22  Alola Dex # (USUM)               123 non-null    float64
23  Melemele Dex # (USUM)            65 non-null     float64
24  Akala Dex # (USUM)              53 non-null     float64
25  Ula'ula Dex # (USUM)            63 non-null     float64
26  Poni Dex #

```

```

(USUM)          41 non-null    float64
 27  Galar Dex #
(SwSh)          66 non-null    float64
 28  Isle of Armor Dex #
(SwSh)  60 non-null    float64
 29  Crown Tund Dex #
(SwSh)          41 non-null    float64
 30  Hisui Dex #
(LA)            54 non-null    float64
 31  Paldea Dex #
(SV)            46 non-null    float64
 32  Kitakami Dex # (SV)      31 non-null    float64
 33  Blueberry Dex # (SV)     51 non-null    float64
 34  HP                      202 non-null    int64
 35  Attack                  202 non-null    int64
 36  Defense                  202 non-null    int64
 37  Special
Attack          202 non-null    int64
 38  Special
Defense         202 non-null    int64
 39  Speed                   202 non-null    int64
 40  Total                   202 non-null    int64
 41  Type I                   202 non-null    object
 42  Type II                  106 non-null    object
 43  Ability I                202 non-null    object
 44  Ability II               126 non-null    object
 45  Hidden Ability           177 non-null    object
 46  Height (ft)              202 non-null    object
 47  Height (m)               202 non-null    float64
 48  Weight (lbs)             202 non-null    float64
 49  Weight (kg)              202 non-null    float64
 50  HP EVs                   23 non-null    float64
 51  Attcak EVs               50 non-null    float64
 52  Defense EVs              35 non-null    float64
 53  Special Attack EVs       38 non-null    float64
 54  Special Defense EVs      22 non-null    float64
 55  Speed EVs                52 non-null    float64
 56  Experience
Value           202 non-null    int64
 57  Catch Rate               202 non-null    int64
 58  Experience
Growth          202 non-null    object
 59  Base Friendship           202 non-null    int64
 60  Pokedex Color            202 non-null    object
 61  Gender Ratio             182 non-null    object
 62  Egg Cycles                202 non-null    int64
 63  Egg Group I               202 non-null    object
 64  Egg Group II              50 non-null    object
 65  Evolution
Method          100 non-null    object
dtypes: float64(40), int64(12), object(14)
memory usage: 104.3+ KB

```

3.6 Estadísticas descriptivas

El método `.describe()` genera un resumen estadístico de las columnas numéricas del DataFrame.

Sintaxis:

`DataFrame.describe()`

EJEMPLO:

```
In [13]: # .describe() muestra una tabla con estadística básica de los datos
pokemon.describe()
```

Out[13]:

	National Dex	Generation	Kanto Dex #(RBYFRLG)	Kanto Dex #(LGPE)	New Dex #(GSC)	Johto Dex #(HGSS)	#
count	202.000000	202.000000	151.000000	151.000000	151.000000	151.000000	47.
mean	76.340594	2.544554	76.000000	76.000000	127.496689	129.377483	102.
std	43.789121	2.702587	43.734045	43.734045	69.608991	71.485173	43.
min	1.000000	1.000000	1.000000	1.000000	10.000000	10.000000	39.
25%	38.325000	1.000000	38.500000	38.500000	72.500000	72.500000	65.
50%	77.050000	1.000000	76.000000	76.000000	125.000000	127.000000	90.
75%	113.750000	4.750000	113.500000	113.500000	182.500000	186.500000	143.
max	151.000000	9.000000	151.000000	151.000000	250.000000	255.000000	185.

8 rows × 52 columns



NOTA: Para las columnas numéricas se calculan estadísticas como:

- count: cantidad de valores.
- mean: media o promedio.
- std: desviación estándar.
- min: valor mínimo.
- 25%: primer cuartil.
- 50%: mediana.
- 75%: tercer cuartil.
- max: valor máximo.

4. Selección de datos

La selección de datos permite extraer únicamente la información que se necesita de un DataFrame. En Pandas es posible seleccionar columnas, filas o valores específicos utilizando diferentes métodos.

- Selección de columnas
- Selección de filas
- Selección con `loc`
- Selección con `iloc`

4.1 Selección de columnas

Para seleccionar una columna de un DataFrame, se escribe el nombre de la columna entre corchetes.

Sintaxis:

```
DataFrame["columna"]
```

EJEMPLO 1:

```
In [14]: # 1 columna
pokemon[["Generation"]]
```

Out[14]:

	Generation
0	1
1	1
2	1
3	6
4	1
...	...
197	1
198	1
199	6
200	6
201	1

	Generation
0	1
1	1
2	1
3	6
4	1
...	...
197	1
198	1
199	6
200	6
201	1

202 rows × 1 columns

También es posible seleccionar varias columnas al mismo tiempo. Para ello se coloca una lista con los nombres de las columnas dentro de los corchetes.

Sintaxis:

```
DataFrame[["columna1", "columna2"]]
```

EJEMPLO 2:

```
In [15]: pokemon[["Generation", "Pokemon\nName"]]
```

```
Out[15]:
```

	Generation	Pokemon\nName
0	1	Bulbasaur
1	1	Ivysaur
2	1	Venusaur
3	6	Venusaur (Mega)
4	1	Charmander
...	...	...
197	1	Dragonite
198	1	Mewtwo
199	6	Mewtwo (Mega X)
200	6	Mewtwo (Mega Y)
201	1	Mew

202 rows × 2 columns

4.2 Selección de filas

Las filas pueden seleccionarse utilizando diferentes métodos. Una forma sencilla es mediante un rango de posiciones utilizando corchetes.

Sintaxis:

```
DataFrame[inicio:fin]
```

EJEMPLO:

```
In [16]: pokemon[10:13]
```

Out[16]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	Nei \n
10	8.0	Wartortle	1	Starter	8.0	8.0	
11	9.0	Blastoise	1	Starter	9.0	9.0	
12	9.1	Blastoise (Mega)	6	Starter, Mega Evolution	NaN	NaN	

3 rows × 66 columns



4.3 Selección de filas por posición `.iloc`

El método `.iloc` permite seleccionar filas de un DataFrame de acuerdo con su posición numérica. Las posiciones comienzan desde 0.

Sintaxis:

`DataFrame.iloc[inicio:fin]`

EJEMPLO:

```
In [17]: # filas
pokemon.iloc[10:13] #integer location, por posición
```

Out[17]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	Nei \n
10	8.0	Wartortle	1	Starter	8.0	8.0	
11	9.0	Blastoise	1	Starter	9.0	9.0	
12	9.1	Blastoise (Mega)	6	Starter, Mega Evolution	NaN	NaN	

3 rows × 66 columns



4.4 Selección de filas por índice `.loc`

El método `.loc` permite seleccionar datos utilizando principalmente las etiquetas del índice y los nombres de las columnas.

Sintaxis:

`DataFrame.loc[[indice1, indice2, indice3]]`

EJEMPLO:

```
In [18]: # filas #por indice
pokemon_copia = pokemon.copy() # para generar una copia y no una referencia del mis
pokemon_copia.loc[[0, 1, 7]]
#pokemon_copia.index = pokemon_copia["name"] #cambio de indice
```

```
Out[18]:
```

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	New #\n()
0	1.0	Bulbasaur	1	Starter	1.0	1.0	2
1	2.0	Ivysaur	1	Starter	2.0	2.0	2
7	6.1	Charizard (Mega X)	6	Starter, Mega Evolution	NaN	NaN	

3 rows × 66 columns



4.5 Restablecer el índice

El método `.reset_index()` permite restablecer el índice de un DataFrame, generando nuevamente un índice numérico consecutivo que comienza desde 0.

Sintaxis:

`DataFrame.reset_index(drop=False)`

- `drop=False`: conserva el índice anterior y lo agrega como una nueva columna.
- `drop=True`: elimina el índice anterior y crea un nuevo índice numérico.

EJEMPLO:

```
In [19]: # reset index
pokemon_copia = pokemon_copia.reset_index(drop=True)
pokemon_copia
```

Out[19]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	↑ #
0	1.0	Bulbasaur	1	Starter	1.0	1.0	
1	2.0	Ivysaur	1	Starter	2.0	2.0	
2	3.0	Venusaur	1	Starter	3.0	3.0	
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	
...	...	...	...	...	...	...	
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

202 rows × 66 columns



4.6 Selección de columnas y filas

En Pandas podemos seleccionar determinadas columnas y posteriormente extraer ciertas filas por su posición utilizando `.iloc`.

Sintaxis:

```
DataFrame[["columna1", "columna2"]].iloc[inicio:fin]
```

EJEMPLO:

```
In [20]: # columnas y filas
pokemon[["Generation", "Pokemon\nName"]].iloc[10:13]
```

```
Out[20]:
```

	Generation	Pokemon\nName
10	1	Wartortle
11	1	Blastoise
12	6	Blastoise (Mega)

5. Filtrado de datos

El filtrado de datos permite seleccionar únicamente las filas de un DataFrame que cumplen una o varias condiciones específicas. En Pandas, las condiciones generan valores True o False, y se conservan únicamente las filas donde la condición es True.

5.1 Filtrar por un valor específico

Para seleccionar las filas que contienen un valor específico en una columna, se puede utilizar el operador de comparación `==`.

Sintaxis:

```
DataFrame[DataFrame["columna"] == valor]
```

EJEMPLO 1:

```
In [21]: # filtro por nombre
filtro_1 = pokemon[pokemon["Pokemon\nName"] == "Charmander"]
filtro_1
```

```
Out[21]:
```

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	New D #\n(GS)
4	4.0	Charmander	1	Starter	4.0	4.0	229

1 rows × 66 columns



NOTA: En este caso, Pandas revisa la columna "Pokemon\nName " y selecciona únicamente las filas donde el nombre sea exactamente "Charmander".

EJEMPLO 2:

```
In [22]: pokemon[pokemon["Experience\nValue"] >= 100]
```

Out[22]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	↕ #
1	2.0	Ivysaur	1	Starter	2.0	2.0	
2	3.0	Venusaur	1	Starter	3.0	3.0	
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
5	5.0	Charmeleon	1	Starter	5.0	5.0	
6	6.0	Charizard	1	Starter	6.0	6.0	
...	...	...	...	...	...	...	
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

132 rows × 66 columns



NOTA: Selecciona todos los Pokémon cuya Experiencia sea mayor o igual a 100.

También puedes utilizar otros operadores de comparación:

== Igual que

!= Diferente de

> Mayor que

< Menor que

>= Mayor o igual que

<= Menor o igual que

5.2 Filtrar por varios valores

El método `.isin()` permite comprobar si los valores de una columna se encuentran dentro de un conjunto de valores especificados.

Sintaxis:

```
DataFrame[DataFrame["columna"].isin([valor1, valor2, valor3])]
```

EJEMPLO:

```
In [23]: # filtrar por nombre
filtro_1 = pokemon[pokemon["Pokemon\Name"].isin(["Charmeleon", "Charizard", "Mew"])
filtro_1
```

```
Out[23]:
```

	National\Nex	Pokemon\Name	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	Ne #\r
5	5.0	Charmeleon	1	Starter	5.0	5.0	
6	6.0	Charizard	1	Starter	6.0	6.0	
201	151.0	Mew	1	Mythical	151.0	151.0	

3 rows × 66 columns



5.3 Filtrar con múltiples condiciones

Pandas también combina dos o más condiciones para realizar filtros más específicos.

Los operadores:

- `&` → significa Y: ambas condiciones deben cumplirse.
- `|` → significa O: al menos una condición debe cumplirse.

Sintaxis con &:

```
DataFrame[ (condicion1) & (condicion2)]
```

EJEMPLO 1:

```
In [24]: # filtrar por rangos "| significa o" "& significa y"
filtro_1 = pokemon[(pokemon["Base Friendship"] >= 100) & (pokemon["Experience\Value"] >= 100)]
filtro_1
```

Out[24]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	Ne \r
45	35.0	Clefairy	1	NaN	35.0	35.0	
46	36.0	Clefable	1	NaN	36.0	36.0	
150	113.0	Chansey	1	NaN	113.0	113.0	
201	151.0	Mew	1	Mythical	151.0	151.0	

4 rows × 66 columns



NOTA: En este caso, únicamente se seleccionan los Pokémon que cumplen las dos condiciones simultáneamente.

Si queremos que se cumpla una condición o la otra, utilizamos |.

Sintaxis con | :

DataFrame[(condicion1) | (condicion2)]

EJEMPLO 2:

```
In [25]: filtro_2 = pokemon[
          (pokemon["Base Friendship"] >= 100) | (pokemon["Experience\nValue"] >= 200)]

filtro_2
```

Out[25]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	I #
	2	3.0	Venusaur	1	Starter	3.0	3.0
	3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN
	6	6.0	Charizard	1	Starter	6.0	6.0
	7	6.1	Charizard (Mega X)	6	Starter, Mega Evolution	NaN	NaN
	8	6.2	Charizard (Mega Y)	6	Starter, Mega Evolution	NaN	NaN
	11	9.0	Blastoise	1	Starter	9.0	9.0
	12	9.1	Blastoise (Mega)	6	Starter, Mega Evolution	NaN	NaN
	19	15.1	Beedrill (Mega)	6	Mega Evolution	NaN	NaN
	22	18.0	Pidgeot	1	NaN	18.0	18.0
	23	18.1	Pidgeot (Mega)	6	Mega Evolution	NaN	NaN
	33	26.0	Raichu	1	NaN	26.0	26.0
	34	26.1	Raichu (Alolan)	7	Alolan Form	NaN	NaN
	41	31.0	Nidoqueen	1	NaN	31.0	31.0
	44	34.0	Nidoking	1	NaN	34.0	34.0
	45	35.0	Clefairy	1	NaN	35.0	35.0
	46	36.0	Clefable	1	NaN	36.0	36.0
	52	40.0	Wigglytuff	1	NaN	40.0	40.0
	57	45.0	Vileplume	1	NaN	45.0	45.0
	81	62.0	Poliwrath	1	NaN	62.0	62.0
	84	65.0	Alakazam	1	NaN	65.0	65.0

	National Dex	Pokemon Name	Generation	Type	Kanto Dex # (RBYFRLG)	Kanto Dex # (LGPE)	Index
85	65.1	Alakazam (Mega)	6	Mega Evolution	NaN	NaN	
88	68.0	Machop	1	NaN	68.0	68.0	
91	71.0	Victreebel	1	NaN	71.0	71.0	
98	76.0	Golem	1	NaN	76.0	76.0	
99	76.1	Golem (Alolan)	7	Alolan Form	NaN	NaN	
107	80.1	Slowbro (Mega)	6	Mega Evolution	NaN	NaN	
125	94.0	Gengar	1	NaN	94.0	94.0	
126	94.1	Gengar (Mega)	6	Mega Evolution	NaN	NaN	
150	113.0	Chansey	1	NaN	113.0	113.0	
153	115.1	Kangaskhan (Mega)	6	Mega Evolution	NaN	NaN	
167	127.1	Pinsir (Mega)	6	Mega Evolution	NaN	NaN	
174	130.1	Gyarados (Mega)	6	Mega Evolution	NaN	NaN	
187	142.1	Aerodactyl (Mega)	6	Mega Evolution	NaN	NaN	
189	144.0	Articuno	1	Legendary	144.0	144.0	
190	144.1	Articuno (Galarian)	8	Legendary, Galarian Form	NaN	NaN	
191	145.0	Zapdos	1	Legendary	145.0	145.0	
192	145.1	Zapdos (Galarian)	8	Legendary, Galarian Form	NaN	NaN	
193	146.0	Moltres	1	Legendary	146.0	146.0	
194	146.1	Moltres (Galarian)	8	Legendary, Galarian Form	NaN	NaN	
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	I #
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

44 rows × 66 columns

NOTA: No es necesario que ambas condiciones se cumplan al mismo tiempo.

6. Modificaciones de DataFrames

Pandas permite crear, insertar y eliminar columnas de un DataFrame. Esto es útil para generar nuevas variables a partir de datos existentes o eliminar información que ya no es necesaria.

6.1 Creación de columnas

Para crear una nueva columna se escribe el nombre de la columna entre corchetes y se asignan los valores que contendrá.

Sintaxis:

- `DataFrame["nueva_columna"] = (DataFrame["columna1"] / DataFrame["columna2"])`

EJEMPLO:

```
In [55]: # creación de columnas
pokemon["Generacion_Aumento"] = pokemon_copia["Generation"] + 2

pokemon.head() ## ¿Recuerdas que hace .head()?
```

Out[55]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	New \n()
0	1.0	Bulbasaur	1	Starter	1.0	1.0	2
1	2.0	Ivysaur	1	Starter	2.0	2.0	2
2	3.0	Venusaur	1	Starter	3.0	3.0	2
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	2

5 rows × 67 columns



NOTA: En este ejemplo se crea la columna "Generacion_Aumento", cuyo valor se obtiene al sumar la Generación del Pokémon (Generation) y el valor de 2.

6.2 Insertar una columna

El método `.insert()` permite agregar una nueva columna en una posición específica del DataFrame.

Sintaxis:

`DataFrame.insert(posicion, "nombre_columna", valores)`

EJEMPLO:

```
In [56]: # insertar columna
pokemon.insert(0, "Amistad_", pokemon["Base Friendship"] / pokemon["Generation"])
pokemon
```

Out[56]:

	Amistad_	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)
0	50.000000	1.0	Bulbasaur	1	Starter	1.0
1	50.000000	2.0	Ivysaur	1	Starter	2.0
2	50.000000	3.0	Venusaur	1	Starter	3.0
3	8.333333	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN
4	50.000000	4.0	Charmander	1	Starter	4.0
...	...	...	...	...	...	...
197	35.000000	149.0	Dragonite	1	Pseudo Legendary	149.0
198	0.000000	150.0	Mewtwo	1	Special	150.0
199	0.000000	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN
200	0.000000	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN
201	100.000000	151.0	Mew	1	Mythical	151.0

202 rows × 68 columns



NOTA: El valor 0 indica que la nueva columna se insertará en la primera posición del DataFrame.

6.3 Eliminar columnas

El método `.drop()` permite eliminar una o varias columnas de un DataFrame.

Sintaxis:

`DataFrame.drop(columns=["columna1", "columna2"])`

EJEMPLO:

```
In [57]: # Eliminar columna
pokemon.drop(columns = ["Amistad_", "Generacion_Aumento"], inplace = True) # inplace
pokemon
```

Out[57]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex #\n(RBYFRLG)	Kanto Dex #\n(LGPE)	↑ #
0	1.0	Bulbasaur	1	Starter	1.0	1.0	
1	2.0	Ivysaur	1	Starter	2.0	2.0	
2	3.0	Venusaur	1	Starter	3.0	3.0	
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	
...	...	...	...	...	...	...	
197	149.0	Dragonite	1	Pseudo Legendary	149.0	149.0	
198	150.0	Mewtwo	1	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1	Mythical	151.0	151.0	

202 rows × 66 columns



NOTA: `inplace=True` indica que la modificación se realizará directamente sobre el DataFrame original.

6.4 Agregar una fila al final

Para agregar una nueva fila al final de un DataFrame, se puede utilizar `.loc` junto con `len()`. La función `len()` obtiene el número de filas actuales y permite utilizar la siguiente posición disponible.

Sintaxis:

```
DataFrame.loc[len(DataFrame)] = {"columna1": valor1, "columna2": valor2}
```

EJEMPLO:

```
In [58]: # agregar filas al final
pokemon.loc[len(pokemon)] = {"pokedex_id": 252,
                              "name": "Nuevo2",
                              "height": 23,
                              "weight": 30,
                              "base_experience": 60,
                              "type_1": "fire",
                              "type_2": "flying",
                              "hp": 50,
                              "attack": 50,
                              "defense": 50,
                              "special_attack": 100,
                              "special_defense": 60,
                              "speed": 30}

pokemon
```

Out[58]:

	National Dex	Pokemon Name	Generation	Type	Kanto Dex # (RBYFRLG)	Kanto Dex # (LGPE)	Ni #
0	1.0	Bulbasaur	1.0	Starter	1.0	1.0	
1	2.0	Ivysaur	1.0	Starter	2.0	2.0	
2	3.0	Venusaur	1.0	Starter	3.0	3.0	
3	3.1	Venusaur (Mega)	6.0	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1.0	Starter	4.0	4.0	
...	...	...	...	...	...	...	...
198	150.0	Mewtwo	1.0	Special	150.0	150.0	
199	150.1	Mewtwo (Mega X)	6.0	Special, Mega Evolution	NaN	NaN	
200	150.2	Mewtwo (Mega Y)	6.0	Special, Mega Evolution	NaN	NaN	
201	151.0	Mew	1.0	Mythical	151.0	151.0	
202	NaN	NaN	NaN	NaN	NaN	NaN	

203 rows × 66 columns



6.5 Agregar varias filas

Cuando se desean agregar varias filas, primero se puede crear un nuevo DataFrame con los registros y posteriormente unirlo al DataFrame original mediante `pd.concat()`.

Sintaxis:

```
nuevo_df = pd.DataFrame(datos)
```

```
DataFrame = pd.concat( [DataFrame, nuevo_df], ignore_index=True)
```

EJEMPLO: Se usara un nuevo archivo tipo `.csv`

```
In [64]: pokemon2 = pd.read_csv("pokemon_stats_2025_1.csv")
pokemon2.head()
```

```
Out[64]:
```

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
0	1	Bulbasaur	7	69	64	grass	poison	45	49
1	2	Ivysaur	10	130	142	grass	poison	60	62
2	3	Venusaur	20	1000	236	grass	poison	80	82
3	4	Charmander	6	85	62	fire	NaN	39	52
4	5	Charmeleon	11	190	142	fire	NaN	58	64

```
In [65]: # agregar filas como dataframe
filas = pd.DataFrame({"pokedex_id": [253, 254],
                      "name": ["Nuevo3", "Nuevo4"],
                      "height": [23, 21],
                      "weight": [30, 40],
                      "base_experience": [60, 50],
                      "type_1": ["fire", "rock"],
                      "type_2": ["flying", "dark"],
                      "hp": [50, 40],
                      "attack": [50, 20],
                      "defense": [50, 35],
                      "special_attack": [100, 32],
                      "special_defense": [60, 25],
                      "speed": [30, 23]})

pokemon2 = pd.concat([pokemon2, filas], ignore_index=True) #ignore_idex hace que el
pokemon2
```

Out[65]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
0	1	Bulbasaur	7	69	64	grass	poison	45	42
1	2	Ivysaur	10	130	142	grass	poison	60	60
2	3	Venusaur	20	1000	236	grass	poison	80	82
3	4	Charmander	6	85	62	fire	NaN	39	52
4	5	Charmeleon	11	190	142	fire	NaN	58	62
...	...	...	...	...	...	...	...	...	...
247	248	Tyranitar	20	2020	270	rock	dark	100	134
248	249	Lugia	52	2160	306	psychic	flying	106	90
249	250	Ho-oh	38	1990	306	fire	flying	106	130
250	253	Nuevo3	23	30	60	fire	flying	50	50
251	254	Nuevo4	21	40	50	rock	dark	40	20

252 rows × 13 columns



6.6 Insertar filas en una posición específica

`pd.concat()` también permite insertar nuevas filas en una determinada posición. Para hacerlo, se divide el DataFrame utilizando `.iloc` y se colocan las nuevas filas entre ambas partes.

Sintaxis:

DataFrame =

```
pd.concat([DataFrame.iloc[:posicion],nuevas_filas,DataFrame.iloc[posicion:]],ignore_index=True)
```

EJEMPLO:

```
In [66]: # insertar nuevas filas como dataframe
filas = pd.DataFrame({"pokedex_id": [255, 256],
                      "name": ["Nuevo5", "Nuevo6"],
                      "height": [23, 21],
                      "weight": [30, 40],
                      "base_experience": [60, 50],
                      "type_1": ["fire", "rock"],
                      "type_2": ["flying", "dark"],
                      "hp": [50, 40],
                      "attack": [50, 20],
                      "defense": [50, 35],
                      "special_attack": [100, 32],
                      "special_defense": [60, 25],
```

```
"speed": [30, 23])
```

```
pokemon2 = pd.concat([pokemon2.iloc[:2], filas, pokemon2.iloc[2:]], ignore_index=True)  
pokemon2
```

Out[66]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
0	1	Bulbasaur	7	69	64	grass	poison	45	49
1	2	Ivysaur	10	130	142	grass	poison	60	62
2	255	Nuevo5	23	30	60	fire	flying	50	50
3	256	Nuevo6	21	40	50	rock	dark	40	20
4	3	Venusaur	20	1000	236	grass	poison	80	82
...	...	...	...	...	...	...	...	...	...
249	248	Tyranitar	20	2020	270	rock	dark	100	134
250	249	Lugia	52	2160	306	psychic	flying	106	90
251	250	Ho-oh	38	1990	306	fire	flying	106	130
252	253	Nuevo3	23	30	60	fire	flying	50	50
253	254	Nuevo4	21	40	50	rock	dark	40	20

254 rows × 13 columns



6.7 Eliminar filas

El método `.drop()` permite eliminar una o varias filas utilizando las etiquetas de su índice.

Sintaxis:

`DataFrame.drop(index=[indice1, indice2])`

EJEMPLO:

```
In [63]: # Eliminar filas  
pokemon2 = pokemon2.drop(index = [1, 2, 3]) # inplace modifica directamente  
pokemon2
```

Out[63]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
0	1	Bulbasaur	7	69	64	grass	poison	45	42
4	3	Venusaur	20	1000	236	grass	poison	80	82
5	4	Charmander	6	85	62	fire	NaN	39	52
6	5	Charmeleon	11	190	142	fire	NaN	58	62
7	6	Charizard	17	905	240	fire	flying	78	84
...	...	...	...	...	...	...	...	...	...
249	248	Tyranitar	20	2020	270	rock	dark	100	130
250	249	Lugia	52	2160	306	psychic	flying	106	90
251	250	Ho-oh	38	1990	306	fire	flying	106	130
252	253	Nuevo3	23	30	60	fire	flying	50	52
253	254	Nuevo4	21	40	50	rock	dark	40	22

251 rows × 13 columns



6.8 Eliminar filas mediante un filtro

También es posible eliminar filas que cumplan una determinada condición. Primero se identifican las filas y posteriormente se utiliza su índice en `.drop()`.

Sintaxis:

```
DataFrame.drop(DataFrame[condicion].index,inplace=True)
```

EJEMPLO:

```
In [67]: # Eliminar por filtro
pokemon2.drop(pokemon2[pokemon2["name"].isin(["Bulbasaur", "Charmeleon"])].index, inplace=True)
pokemon2
```

Out[67]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
1	2	Ivysaur	10	130	142	grass	poison	60	62
2	255	Nuevo5	23	30	60	fire	flying	50	55
3	256	Nuevo6	21	40	50	rock	dark	40	20
4	3	Venusaur	20	1000	236	grass	poison	80	85
5	4	Charmander	6	85	62	fire	NaN	39	52
...	...	...	...	...	...	...	...	...	...
249	248	Tyranitar	20	2020	270	rock	dark	100	135
250	249	Lugia	52	2160	306	psychic	flying	106	95
251	250	Ho-oh	38	1990	306	fire	flying	106	130
252	253	Nuevo3	23	30	60	fire	flying	50	55
253	254	Nuevo4	21	40	50	rock	dark	40	20

252 rows × 13 columns



6.9 Eliminar filas con múltiples condiciones

Los filtros también pueden utilizarse para eliminar registros que cumplan condiciones numéricas. F Sintaxis:

```
filtro = DataFrame[(DataFrame["columna1"] >= valor) |(DataFrame["columna2"] >= valor)]
```

Se pueden ocupar diferentes operadores como:

`==` Igual que

`!=` Diferente de

`>` Mayor que

`<` Menor que

`>=` Mayor o igual que

`<=` Menor o igual que

EJEMPLO:

```
In [68]: # Eliminar por rangos
filtro_1 = pokemon2[(pokemon2["attack"] >= 100) | (pokemon2["defense"] >= 100)]
```

```
pokemon2.drop(filtro_1.index, inplace=True)
pokemon2
```

Out[68]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
1	2	Ivysaur	10	130	142	grass	poison	60	60
2	255	Nuevo5	23	30	60	fire	flying	50	50
3	256	Nuevo6	21	40	50	rock	dark	40	20
4	3	Venusaur	20	1000	236	grass	poison	80	80
5	4	Charmander	6	85	62	fire	NaN	39	50
...	...	...	...	...	...	...	...	...	...
244	243	Raikou	19	1780	261	electric	NaN	90	80
247	246	Larvitar	6	720	60	rock	ground	50	60
248	247	Pupitar	12	1520	144	rock	ground	70	80
252	253	Nuevo3	23	30	60	fire	flying	50	50
253	254	Nuevo4	21	40	50	rock	dark	40	20

187 rows × 13 columns



NOTA: Se eliminan los Pokémon cuyo ataque sea mayor o igual a 100 O cuya defensa sea mayor o igual a 100.

6.10 Renombrar columnas

El método `.rename()` permite cambiar el nombre de una o varias columnas de un DataFrame.

Sintaxis:

```
DataFrame.rename( columns={ "nombre_anterior": "nombre_nuevo" }, inplace=True )
```

EJEMPLO:

```
In [69]: # cambiar el nombre a una o varias columnas
pokemon2.rename(columns = {"name": "nombre", "type_1": "tipo_1"}, inplace = True)
pokemon2 = pokemon2.reset_index(drop=True)
pokemon2
```

Out[69]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack
0	2	Ivysaur	10	130	142	grass	poison	60	60
1	255	Nuevo5	23	30	60	fire	flying	50	50
2	256	Nuevo6	21	40	50	rock	dark	40	20
3	3	Venusaur	20	1000	236	grass	poison	80	80
4	4	Charmander	6	85	62	fire	NaN	39	50
...	...	...	...	...	...	...	...	...	...
182	243	Raikou	19	1780	261	electric	NaN	90	80
183	246	Larvitar	6	720	60	rock	ground	50	60
184	247	Pupitar	12	1520	144	rock	ground	70	80
185	253	Nuevo3	23	30	60	fire	flying	50	50
186	254	Nuevo4	21	40	50	rock	dark	40	20

187 rows × 13 columns



6.11 Identificar valores nulos

El método `.isna()` permite identificar qué valores del DataFrame son nulos.

Sintaxis:

`DataFrame.isna()`

EJEMPLO:

```
In [70]: # manejo de nulos
pokemon2.isna()
```

Out[70]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	True	False	False
...	...	...	...	...	...	...	...	...	...
182	False	False	False	False	False	False	True	False	False
183	False	False	False	False	False	False	False	False	False
184	False	False	False	False	False	False	False	False	False
185	False	False	False	False	False	False	False	False	False
186	False	False	False	False	False	False	False	False	False

187 rows × 13 columns



6.12 Contar valores nulos

Para conocer la cantidad de valores nulos presentes en cada columna se puede combinar

`.isna()` con `.sum()`.

Sintaxis:

`DataFrame.isna().sum()`

EJEMPLO:

```
In [71]: # manejo de nulos
pokemon.isna().sum() #resumen de valores nulos
```

```
Out[71]: National\nDex          1
Pokemon\nName          1
Generation              1
Type                  127
Kanto Dex #\n(RBYFRLG)   52
...
Gender Ratio          21
Egg Cycles             1
Egg Group I            1
Egg Group II          153
Evolution\nMethod       103
Length: 66, dtype: int64
```

6.13 Eliminar valores nulos

El método `.dropna()` permite eliminar registros que contienen valores nulos.

Sintaxis:

`DataFrame.dropna(inplace=True)`

EJEMPLO:

```
In [72]: # eliminar nulos (elimina toda la fila donde se encuentre algun nulo)
pokemon2.dropna(inplace=True)
#pokemon.reset_index(drop = True)
pokemon2.isna().sum()#resumen de valores nulos
```

```
Out[72]: pokedex_id      0
nombre      0
height      0
weight      0
base_experience  0
tipo_1      0
type_2      0
hp          0
attack      0
defense     0
special_attack  0
special_defense  0
speed       0
dtype: int64
```

6.14 Restablecer el índice

Después de eliminar filas, el índice puede quedar discontinuo. El método `.reset_index()` permite generar nuevamente un índice consecutivo.

Sintaxis:

`DataFrame.reset_index(drop=True)`

EJEMPLO:

```
In [74]: # reset index
pokemon2 = pokemon2.reset_index(drop = True)
pokemon2
```

Out[74]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack
0	2	Ivysaur	10	130	142	grass	poison	60	62
1	255	Nuevo5	23	30	60	fire	flying	50	50
2	256	Nuevo6	21	40	50	rock	dark	40	20
3	3	Venusaur	20	1000	236	grass	poison	80	82
4	6	Charizard	17	905	240	fire	flying	78	84
...	...	...	...	...	...	...	...	...	...
82	238	Smoochum	4	60	61	ice	psychic	45	30
83	246	Larvitar	6	720	60	rock	ground	50	64
84	247	Pupitar	12	1520	144	rock	ground	70	84
85	253	Nuevo3	23	30	60	fire	flying	50	50
86	254	Nuevo4	21	40	50	rock	dark	40	20

87 rows × 13 columns



6.15 Reemplazar valores

El método `.replace()` permite sustituir uno o varios valores existentes por nuevos valores.

Sintaxis:

```
DataFrame["columna"].replace([valor1, valor2],[nuevo1, nuevo2])
```

EJEMPLO:

```
In [78]: # reemplazar valores
pokemon2["nombre"] = pokemon2["nombre"].replace(["Charizard", "Weedle"], ["CHARIZAR", "WEEDLE"])
pokemon2.head(7)
```

Out[78]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack	...
0	2	Ivysaur	10	130	142	grass	poison	60	62	
1	255	Nuevo5	23	30	60	fire	flying	50	50	
2	256	Nuevo6	21	40	50	rock	dark	40	20	
3	3	Venusaur	20	1000	236	grass	poison	80	82	
4	6	CHARIZARD	17	905	240	fire	flying	78	84	
5	12	Butterfree	11	320	178	bug	flying	60	45	
6	13	WEEDLE	3	32	39	bug	poison	40	35	

6.16 Reemplazar valores utilizando un diccionario

También se puede utilizar un diccionario para indicar qué valor será reemplazado y cuál será su sustituto.

Sintaxis:

```
DataFrame["columna"].replace({valor_anterior: valor_nuevo})
```

EJEMPLO:

```
In [80]: # reemplazar valores con diccionarios
pokemon2["nombre"] = pokemon2["nombre"].replace({"CHARIZARD": "Charizard", "WEEDLE": "Weedle"})
pokemon2.head(7)
```

Out[80]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack	de
0	2	Ivysaur	10	130	142	grass	poison	60	62	
1	255	Nuevo5	23	30	60	fire	flying	50	50	
2	256	Nuevo6	21	40	50	rock	dark	40	20	
3	3	Venusaur	20	1000	236	grass	poison	80	82	
4	6	Charizard	17	905	240	fire	flying	78	84	
5	12	Butterfree	11	320	178	bug	flying	60	45	
6	13	Weedle	3	32	39	bug	poison	40	35	

NOTA: `.replace()` permite sustituir valores utilizando diferentes estructuras. Con listas, los valores se relacionan según su posición; con un diccionario, se establece explícitamente

la relación valor_anterior: valor_nuevo. El uso de diccionarios suele ser más claro cuando se realizan múltiples reemplazos.

7. Ordenamiento de DataFrame

El ordenamiento de datos permite organizar las filas de un DataFrame para facilitar su visualización, comparación y análisis. Los datos pueden ordenarse de forma ascendente o descendente, ya sea utilizando los valores de una o varias columnas o las etiquetas del índice.

Para realizar estas operaciones, Pandas proporciona principalmente los métodos `.sort_values()` y `.sort_index()`.

7.1 Ordenar por valores

El método `.sort_values()` permite ordenar las filas de un DataFrame de acuerdo con los valores de una o varias columnas. Los datos pueden organizarse en orden ascendente o descendente.

Sintaxis 1:

```
DataFrame.sort_values(by="columna",ascending=True)
```

- `ascending=True`: ordena el índice de forma ascendente

EJEMPLO:

```
In [82]: # ordenar en orden descendente
pokemon2.sort_values("attack", ascending = False)
```

Out[82]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack	
41	103	Exeggutor	20	1200	186	grass	psychic	95	95	
81	230	Kingdra	18	1520	243	water	dragon	75	95	
26	62	Poliwrath	13	540	230	water	fighting	90	95	
73	211	Qwilfish	5	39	88	water	poison	65	95	
74	215	Sneasel	9	280	86	dark	ice	55	95	
...	...	...	...	...	...	...	...	...	...	
7	14	Kakuna	6	100	72	bug	poison	45	25	
2	256	Nuevo6	21	40	50	rock	dark	40	20	
51	165	Ledyba	10	108	53	bug	flying	40	20	
62	183	Marill	4	85	88	water	fairy	70	20	
86	254	Nuevo4	21	40	50	rock	dark	40	20	

87 rows × 13 columns



Sintaxis 2:

DataFrame.sort_values(by="columna",ascending=False)

- ascending=False: ordena el índice de forma descendente.

EJEMPLO:

```
In [ ]: # ordenar en orden ascendente
pokemon.sort_values("nombre", ascending = True)
```

```
Out[ ]:
```

	attack_speed	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2
51	2.250000	168	Ariados	11	335	140	bug	poison
60	1.000000	184	Azumarill	8	285	189	water	fairy
5	1.200000	15	Beedrill	10	295	178	bug	poison
24	1.875000	69	Bellsprout	7	40	60	grass	poison
2	0.642857	12	Butterfree	11	320	178	bug	flying
...	...	...	...	...	...	...	...	...
65	3.000000	194	Wooper	4	85	42	water	ground
58	0.789474	178	Xatu	15	150	165	psychic	flying
64	0.684211	193	Yanma	12	380	78	bug	flying
45	0.900000	145	Zapdos	16	526	261	electric	flying
14	0.818182	41	Zubat	8	75	49	poison	flying

82 rows × 14 columns



7.3 Ordenar por índice

El método `.sort_index()` permite ordenar las filas de un DataFrame de acuerdo con las etiquetas de su índice.

Sintaxis 1:

```
DataFrame.sort_index(ascending=True)
```

- `ascending=True`: ordena el índice de forma ascendente.

EJEMPLO:

```
In [91]: # sort index
pokemon2.sort_index(ascending=True)
```

Out[91]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack
0	2	Ivysaur	10	130	142	grass	poison	60	62
1	255	Nuevo5	23	30	60	fire	flying	50	50
2	256	Nuevo6	21	40	50	rock	dark	40	20
3	3	Venusaur	20	1000	236	grass	poison	80	82
4	6	Charizard	17	905	240	fire	flying	78	84
...	...	...	...	...	...	...	...	...	...
82	238	Smoochum	4	60	61	ice	psychic	45	30
83	246	Larvitar	6	720	60	rock	ground	50	64
84	247	Pupitar	12	1520	144	rock	ground	70	84
85	253	Nuevo3	23	30	60	fire	flying	50	50
86	254	Nuevo4	21	40	50	rock	dark	40	20

87 rows × 13 columns



Sintaxis 2:

DataFrame.sort_index(ascending=False)

- ascending=False: ordena el índice de forma descendente.

EJEMPLO:

```
In [90]: # sort index
pokemon2.sort_index(ascending=False)
```

Out[90]:

	pokedex_id	nombre	height	weight	base_experience	tipo_1	type_2	hp	attack
86	254	Nuevo4	21	40	50	rock	dark	40	20
85	253	Nuevo3	23	30	60	fire	flying	50	50
84	247	Pupitar	12	1520	144	rock	ground	70	84
83	246	Larvitar	6	720	60	rock	ground	50	64
82	238	Smoochum	4	60	61	ice	psychic	45	30
...	...	...	...	...	...	...	...	...	...
4	6	Charizard	17	905	240	fire	flying	78	84
3	3	Venusaur	20	1000	236	grass	poison	80	82
2	256	Nuevo6	21	40	50	rock	dark	40	20
1	255	Nuevo5	23	30	60	fire	flying	50	50
0	2	Ivysaur	10	130	142	grass	poison	60	62

87 rows × 13 columns



8. Operaciones con DataFrames

Pandas permite realizar diferentes operaciones matemáticas y estadísticas sobre los datos almacenados en un DataFrame. Estas operaciones son útiles para analizar variables numéricas y obtener información como el promedio, mediana, desviación estándar, valores mínimos y máximos, así como la cantidad de datos disponibles.

8.1 Operaciones Aritméticas

Es posible realizar operaciones matemáticas directamente sobre los valores de una columna.

Sintaxis:

`DataFrame["columna"].iloc[posicion] operador valor`

Operadores: + Suma , - Resta, * Multiplicación, / División, ** Potencia

EJEMPLO:

```
In [101... print(pokemon2["attack"].iloc[0]*2)
```

NOTA: `.iloc[0]` selecciona el primer valor de esa columna.

8.2 Media

Calcula el promedio de los valores de una columna.

Sintaxis:

`DataFrame["columna"].mean()`

EJEMPLO:

```
In [102... print(pokemon2["attack"].mean())  
62.05747126436781
```

8.3 Mediana

Calcula la mediana, es decir, el valor central de los datos después de ordenarlos.

Sintaxis:

`DataFrame["columna"].median()`

EJEMPLO:

```
In [103... print(pokemon2["attack"].median())  
60.0
```

8.4 Desviación estándar

Calcula la desviación estándar, que permite conocer qué tan dispersos se encuentran los valores respecto a su media.

Sintaxis:

`DataFrame["columna"].std()`

EJEMPLO:

```
In [104... print(pokemon2["attack"].std())  
21.973745603713606
```

8.5 Valor mínimo

Obtiene el valor más pequeño de una columna.

Sintaxis:

DataFrame["columna"].min()

EJEMPLO:

```
In [105... print(pokemon2["attack"].min())
```

20

8.6 Valor máximo

Obtiene el valor más grande de una columna.

Sintaxis:

DataFrame["columna"].max()

EJEMPLO:

```
In [106... print(pokemon2["attack"].max())
```

95

8.7 Conteo de valores

El método `.count()` permite contar la cantidad de valores no nulos presentes en una columna.

Sintaxis:

DataFrame["columna"].count()

EJEMPLO:

```
In [100... print(pokemon2["attack"].count())
```

87

9. Agrupaciones

Las agrupaciones permiten organizar los datos de un DataFrame en grupos de acuerdo con los valores de una o varias columnas. Una vez agrupados los datos, es posible aplicar operaciones estadísticas como conteo, media, suma, mínimo o máximo a cada grupo.

Sintaxis:

```
DataFrame.groupby("columna_agrupar")["columna_analizar"].operacion()
```

- `.operacion()` : indica el cálculo que se realizará

EJEMPLO 1:

```
In [108... # agrupaciones
pokemon2.groupby("tipo_1")["nombre"].count()
```

```
Out[108... tipo_1
bug          13
dark          4
electric      3
fairy          1
fire           3
ghost          3
grass         12
ground         1
ice            4
normal        13
poison         4
psychic        3
rock           5
water         18
Name: nombre, dtype: int64
```

NOTA: Esto permite conocer cuántos Pokémon existen de cada tipo en el DataFrame.

EJEMPLO 2:

```
In [ ]: # agrupaciones
pokemon.groupby("tipo_1")["attack"].mean()
```

Out[]:

	attack
tipo_1	
bug	57.692308
dark	82.500000
electric	61.666667
fairy	40.000000
fire	84.000000
ghost	50.000000
grass	64.727273
ground	85.000000
ice	46.250000
normal	62.692308
poison	76.750000
psychic	56.666667
rock	76.000000
water	64.222222

dtype: float64

NOTA: Calcula el promedio de ataque de los Pokémon pertenecientes a cada tipo.

10. Unión de DataFrames

La unión de DataFrames permite combinar información proveniente de dos conjuntos de datos diferentes utilizando una columna en común. Esta operación es útil cuando la información relacionada se encuentra distribuida en distintos archivos o tablas.

La función `pd.merge()` permite unir dos DataFrames utilizando los valores coincidentes de una columna en común.

Sintaxis:

```
pd.merge( DataFrame1,DataFrame2,on="columna_comun")
```

EJEMPLO:

Primero se carga el segundo archivo que contiene la información adicional.

```
In [110... # datos para merge
region = pd.read_csv("pokemon_stats_2025_2.csv")
region
```

```
Out[110...      name  region
0      Abra  Kanto
1  Aerodactyl  Kanto
2      Aipom  Johto
3  Alakazam  Kanto
4  Ampharos  Johto
...      ...    ...
245  Wooper  Johto
246      Xatu  Johto
247  Yanma  Johto
248  Zapdos  Kanto
249  Zubat  Kanto
```

250 rows × 2 columns

Para realizar la unión, ambos DataFrames deben tener una columna que permita relacionar la información. La columna en común será **name**

```
In [112... # merge
pokemon2.rename(columns = {"nombre": "name"}, inplace = True) # cambiando nombre de
pokemon_region = pd.merge(pokemon2, region, on = "name")
pokemon_region
```

Out[112...

	pokedex_id	name	height	weight	base_experience	tipo_1	type_2	hp	attack
0	2	Ivysaur	10	130	142	grass	poison	60	62
1	3	Venusaur	20	1000	236	grass	poison	80	82
2	6	Charizard	17	905	240	fire	flying	78	84
3	12	Butterfree	11	320	178	bug	flying	60	45
4	13	Weedle	3	32	39	bug	poison	40	35
...	...	...	...	...	...	...	...	...	...
78	229	Houndoom	14	350	175	dark	fire	75	90
79	230	Kingdra	18	1520	243	water	dragon	75	95
80	238	Smoochum	4	60	61	ice	psychic	45	30
81	246	Larvitar	6	720	60	rock	ground	50	64
82	247	Pupitar	12	1520	144	rock	ground	70	84

83 rows × 14 columns



NOTA: Entonces "name" funciona como la columna de unión: Pandas busca los nombres que coinciden en pokemon y region y reúne la información de ambos DataFrames en uno nuevo.

11. Lectura de archivos Excel

Pandas permite leer e importar datos almacenados en archivos de Excel `.xlsx` y convertirlos en un DataFrame, facilitando su posterior exploración, manipulación y análisis.

11.1 Archivos desde GitHub

En Python se puede manejar archivos de texto plano `.txt` a través de herramientas integradas que facilitan leer y editar archivos de manera controlada.

Sintaxis:

```
pd.read_excel(ruta_o_url)
```

EJEMPLO:

```
In [7]: import pandas as pd
url = "https://raw.githubusercontent.com/Moises13r/Rahxion_figures/23fc28fa1bc06639
```

```
file1 = pd.read_excel(url)
file1.head()
```

Out[7]:

	Name	Country	Sex	Math	Physics	Chemistry	Biology	History	English	Programmr
0	Maria Smith	Canada	M	74	68	66	94	65	97	
1	Alex Smith	Mexico	M	74	92	98	61	95	72	
2	David Johnson	Brazil	F	60	70	87	81	77	69	
3	Sofia Smith	Mexico	F	66	82	82	98	76	62	
4	Carlos Smith	Brazil	M	95	78	100	99	83	96	



11.2 Archivos desde local

En Jupyter Notebook, Pandas permite leer archivos de Excel almacenados localmente en la computadora mediante la función `pd.read_excel()`.

Si el archivo de Excel se encuentra en la misma carpeta que el Notebook, únicamente es necesario indicar el nombre del archivo.

Sintaxis:

```
pd.read_excel("nombre_archivo.xlsx")
```

EJEMPLO:

```
In [11]: import pandas as pd
pokemon = pd.read_excel("N_Pokedex_Gen_9.xlsx")

pokemon.head()
```

Out[11]:

	National\nDex	Pokemon\nName	Generation	Type	Kanto Dex \n(RBYFRLG)	Kanto Dex \n(LGPE)	New \n()
0	1.0	Bulbasaur	1	Starter	1.0	1.0	2
1	2.0	Ivysaur	1	Starter	2.0	2.0	2
2	3.0	Venusaur	1	Starter	3.0	3.0	2
3	3.1	Venusaur (Mega)	6	Starter, Mega Evolution	NaN	NaN	
4	4.0	Charmander	1	Starter	4.0	4.0	2

5 rows × 66 columns



12. Lectura de archivos CSV

12.1 Archivo desde GitHub

Pandas permite leer archivos CSV `.csv` almacenados en GitHub directamente desde su dirección URL. Convierte la información del archivo en un DataFrame.

Sintaxis:

```
pd.read_csv(url)
```

EJEMPLO:

```
In [15]: import pandas as pd
url = "https://raw.githubusercontent.com/Moises13r/Rahxion_figures/23fc28fa1bc06639
file3 = pd.read_csv(url)
file3.head()
```

Out[15]:

	Name	Country	Sex	Math	Physics	Chemistry	Biology	History	English	Programmr
0	Maria Smith	Canada	M	74	68	66	94	65	97	
1	Alex Smith	Mexico	M	74	92	98	61	95	72	
2	David Johnson	Brazil	F	60	70	87	81	77	69	
3	Sofia Smith	Mexico	F	66	82	82	98	76	62	
4	Carlos Smith	Brazil	M	95	78	100	99	83	96	

12.2 Archivo desde local

En Jupyter Notebook, Pandas permite leer archivos CSV `.csv` almacenados localmente en la computadora. Si el archivo CSV se encuentra en la misma carpeta que el Notebook, únicamente es necesario indicar el nombre del archivo.

Sintaxis:

```
pd.read_csv("nombre_archivo.csv")
```

EJEMPLO:

```
In [19]: import pandas as pd
file5 = pd.read_csv("pokemon_stats_2025_1.csv")
file5.head()
```

Out[19]:

	pokedex_id	name	height	weight	base_experience	type_1	type_2	hp	attack
0	1	Bulbasaur	7	69	64	grass	poison	45	49
1	2	Ivysaur	10	130	142	grass	poison	60	62
2	3	Venusaur	20	1000	236	grass	poison	80	82
3	4	Charmander	6	85	62	fire	NaN	39	52
4	5	Charmeleon	11	190	142	fire	NaN	58	64

13. Análisis exploratorio de datos

13.1 Tipo de clase

La función `type()` permite identificar el tipo de objeto almacenado en una variable.

Sintaxis:

`type(variable)`

EJEMPLO:

```
In [22]: data=file1
print(type(data))
```

```
<class 'pandas.core.frame.DataFrame'>
```

NOTA: El resultado `pandas.core.frame.DataFrame` indica que los datos importados fueron almacenados correctamente como un `DataFrame` de `Pandas`.

13.2 Realizar una copia del archivo original

El método `.copy()` permite crear una copia independiente de un `DataFrame`. Esto es útil cuando se desea limpiar, modificar o transformar los datos sin alterar el `DataFrame` original.

Sintaxis:

`nuevo_dataframe = dataframe_original.copy()`

EJEMPLO:

```
In [23]: data_clean=data.copy()
data_clean.describe()
```

```
Out[23]:
```

	Math	Physics	Chemistry	Biology	History	English	Programmi
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.0000
mean	80.905000	79.200000	79.590000	81.050000	80.930000	80.455000	81.4000
std	11.679384	11.872438	11.567234	11.835451	11.746275	11.655782	11.7104
min	60.000000	60.000000	60.000000	60.000000	60.000000	60.000000	60.0000
25%	70.000000	68.750000	69.750000	71.000000	69.000000	70.000000	70.7500
50%	81.000000	79.000000	78.500000	82.500000	82.000000	81.000000	83.0000
75%	91.000000	89.000000	89.250000	91.250000	90.250000	90.000000	91.0000
max	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.0000



14. Visualización de datos

14.1 Datos bivariados

Los datos bivariados son aquellos en los que se analizan dos variables al mismo tiempo con el objetivo de observar si existe alguna relación o comportamiento entre ellas.

Una forma sencilla de representar dos variables numéricas es mediante un gráfico de dispersión, donde cada punto representa una observación del conjunto de datos.

Sintaxis:

```
plt.plot(x, y, "bo")
```

- x: contiene los valores que se representan en el eje X.
- y: contiene los valores que se representan en el eje Y.
- b: blue (azul).
- o: Puntos circulares.

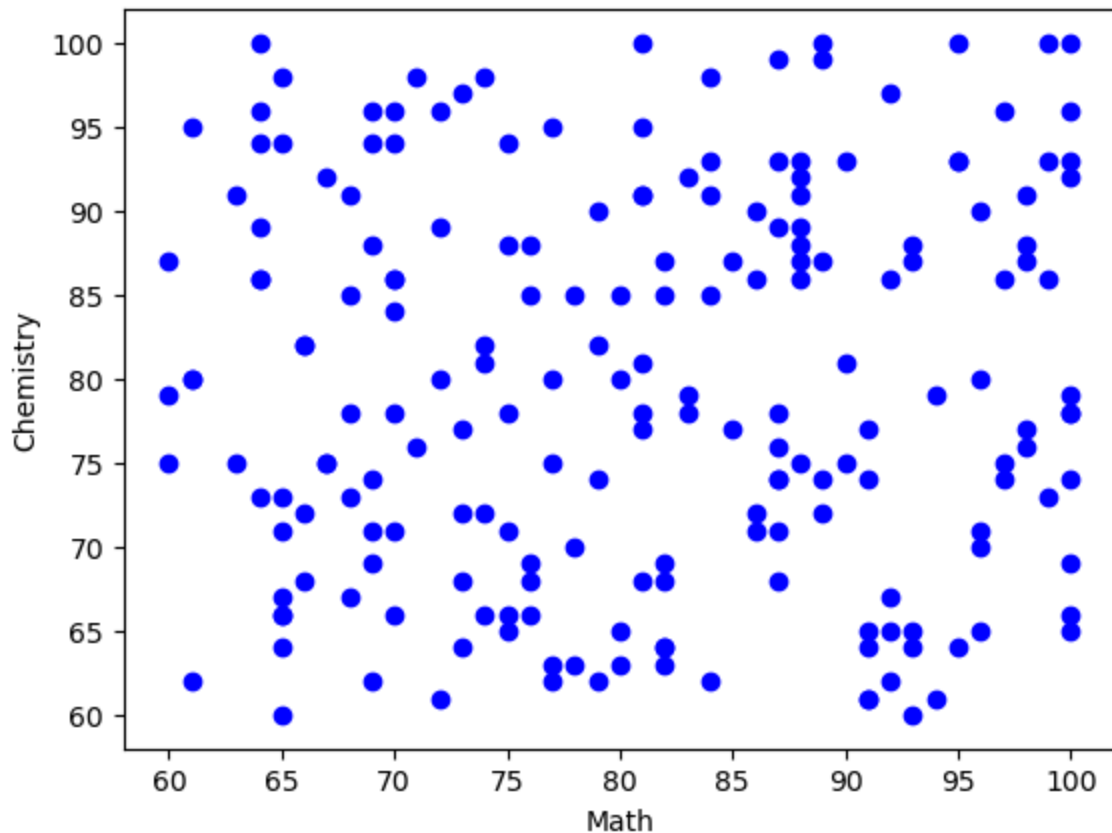
EJEMPLO: Compara la variable Math con la variable Chemistry

```
In [25]: import matplotlib.pyplot as plt
```

NOTA: Antes es necesario importar la biblioteca Matplotlib

```
In [26]: plt.plot(data.iloc[:,3],data.iloc[:,5],"bo")  
plt.xlabel("Math")  
plt.ylabel("Chemistry")
```

```
Out[26]: Text(0, 0.5, 'Chemistry')
```



14.2 Histograma

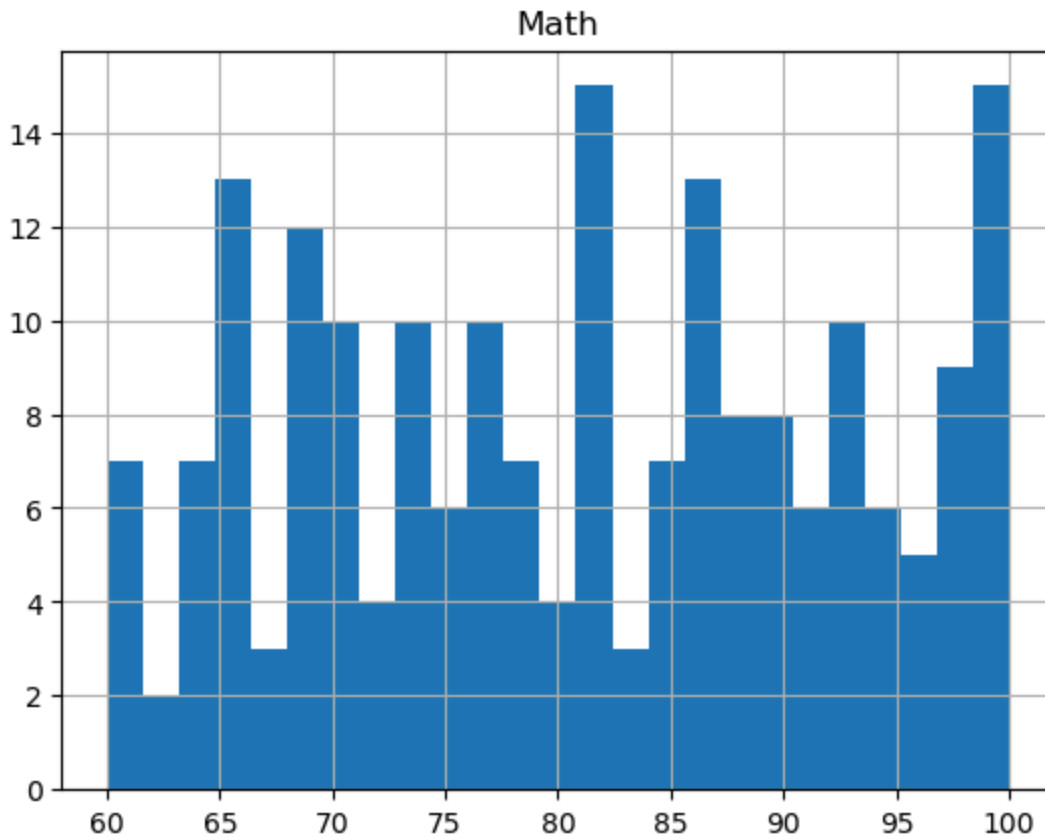
Un histograma es una representación gráfica que permite observar la distribución de una variable numérica. Los valores se agrupan en intervalos y la altura de cada barra indica la frecuencia, es decir, cuántos datos se encuentran dentro de cada intervalo.

Sintaxis: `DataFrame.hist(column="nombre_columna", bins=numero_intervalos)`

EJEMPLO:

```
In [27]: data_clean.hist("Math", bins=25)
```

```
Out[27]: array([[<Axes: title={'center': 'Math'}>]], dtype=object)
```



15. Exportación de DataFrames

Después de realizar la limpieza, modificación o análisis de los datos, Pandas permite exportar un DataFrame a diferentes formatos de archivo para almacenar los resultados y utilizarlos posteriormente en otros programas.

Entre los formatos más utilizados se encuentran:

- Excel (.xlsx)
- CSV (.csv)
- Archivos de texto (.txt)

15.1 Descargar .xlsx

Para guardar un DataFrame como un archivo de Excel se utiliza el método `.to_excel()`.

Sintaxis:

```
DataFrame.to_excel("nombre_archivo.xlsx", index=False)
```

EJEMPLO:

```
In [113]: # Exportar DataFrame a Excel indicando el nombre de la hoja
data_clean.to_excel("datos_limpios.xlsx", index=False)
```

NOTA: Para conocer la carpeta donde Jupyter Notebook está trabajando y donde se guardarán los archivos exportados cuando no se especifica una ruta, se puede utilizar el módulo `os` junto con la función `os.getcwd()`.

```
In [114]: import os

os.getcwd()
```

```
Out[114]: 'C:\\Users\\Ximme\\TEMAS_SELECTOS_2027_1'
```

15.2 Descargar .csv

Para guardar un DataFrame como un archivo CSV se utiliza el método `.to_csv()`.

Sintaxis:

```
DataFrame.to_csv("nombre_archivo.csv", index=False)
```

EJEMPLO:

```
In [33]: data_clean.to_csv("datos_limpios.csv", index=False)
```

15.3 Descargar .txt

Pandas también permite guardar un DataFrame como un archivo de texto. Para ello se puede utilizar nuevamente el método `.to_csv()`, pero indicando la extensión `.txt` y el separador deseado.

Sintaxis:

```
DataFrame.to_csv("nombre_archivo.txt", sep="separador", index=False)
```

EJEMPLO:

```
In [34]: data_clean.to_csv("datos_limpios.txt", sep="\t", index=False)
```

NOTA: `sep="\t"` indica que las columnas estarán separadas mediante una tabulación.

16. Conclusiones

- Los DataFrames (principalmente con pandas) permiten organizar grandes volúmenes de datos en estructuras tabulares claras, haciendo más sencillo filtrar, ordenar, agrupar y analizar información.
- El manejo adecuado de DataFrames permite detectar valores nulos, duplicados o inconsistentes, lo cual es clave para garantizar la calidad de los datos antes de cualquier análisis o modelo.
- Gracias a sus funciones integradas, se reducen muchas líneas de código y se agilizan tareas comunes, permitiendo enfocarse más en la interpretación de resultados que en la programación básica.
- La mayoría de los modelos y bibliotecas de análisis trabajan directamente con DataFrames, por lo que dominarlos es esencial para transformar datos en información útil y entrenar modelos confiables.
- Los DataFrames se integran fácilmente con archivos CSV, Excel, bases de datos y librerías de visualización, lo que los convierte en un puente clave entre distintas fuentes de datos y procesos analíticos.